

AWS Master Data Engineer

6-Month Exhaustive Training & Certification Program

Advanced Architecture, Hands-on Labs, and MLOps

Abstract

This 24-week program is designed to transform you from a practitioner to a Master Data Engineer on Amazon Web Services (AWS). It progressively builds from core S3 data lakes to advanced AWS Lake Formation governance, integrating daily theory, weekly service-specific mini-projects, architectural use case analyses, and massive monthly Capstone projects. A tracking timetable is included at the end.

Contents

1 Milestone 1: Core Data Infrastructure & Storage (Month 1)	2
2 Milestone 2: Data Warehousing & Analytics (Month 2)	5
3 Milestone 3: Batch Processing & ETL Orchestration (Month 3)	10
4 Milestone 4: Streaming & Real-Time Analytics (Month 4)	15
5 Milestone 5: Machine Learning Operationalization (Month 5)	20
6 Milestone 6: Security, Governance & Exam Prep (Month 6)	24
7 Appendix A: Glossary	30
8 Appendix B: Assessment & Grading Model	31
9 Appendix C: Interview Preparation	31
10 Appendix D: Self-Check Answer Key	33
11 Appendix E: Datasets & Project Data	37
12 Appendix F: Portfolio & Presentation Guide	37
13 6-Month Progress Tracking Timetable	39

Prerequisites & How to Use This Syllabus

- **Prerequisite skills:** intermediate Python (Boto3), solid SQL, Git workflows (branching, pull requests), Linux/CLI basics, and Docker fundamentals.
- **Weekly cadence:** approximately 10–12 hours per week — Monday to Wednesday for theory, Thursday for the hands-on project, Friday for the architecture use case.
- **Deliverable expectation:** every capstone must produce a git repository, an architecture diagram, a cost estimate, and a security review checklist, per the Acceptance Criteria boxes that follow each capstone.
- **Assessment:** the 24-week tracking timetable at the end of this document doubles as the assessment checklist — check off each week only when its project and deliverables are complete.

Environment setup checklist:

- Install Python 3.x, Git, Docker Desktop, and VS Code.
- Install the AWS CLI v2 (plus the Session Manager plugin); verify with `aws sts get-caller-identity`.
- Create an AWS free-tier account with guardrails: MFA on the root user, an IAM Identity Center admin role for daily work, and an AWS Budgets cost alert.
- Clone the course repository structure: `data/`, `src/`, `tests/`, `infra/`, `docs/`.
- Configure credentials via environment variables or named CLI profiles — never hardcode keys in code or notebooks.

1 Milestone 1: Core Data Infrastructure & Storage (Month 1)

Objective: Master the foundational storage options, relational databases, and NoSQL.

Learning outcomes: design S3 storage-class and lifecycle strategies, deploy and fail over Aurora Serverless clusters, model single-table DynamoDB schemas with GSIs, replicate relational data to S3 with DMS, and select the right cache tier for latency-sensitive workloads.

Week 1: Amazon S3 & Storage Foundations

Outcomes: compare S3 storage classes and retrieval trade-offs, configure lifecycle policies and versioning, select the right migration service (DataSync vs. Snow), and wire event notifications to SQS.

- **Monday:** S3 Architecture, Storage Classes (Standard, IA, Glacier Flexible Retrieval/Deep Archive).
- **Tuesday:** Lifecycle Policies, Versioning, Object Lock, and S3 Replication.
- **Wednesday:** AWS DataSync, Storage Gateway, and AWS Snow Family.
- **Thursday: Hands-on Project:** Deploy an S3 bucket via AWS CLI, configure a 30-day lifecycle transition to Glacier, and set up event notifications to SQS.
- **Friday: Use Case & Architecture:** Design a multi-region, compliant disaster recovery storage architecture for a healthcare provider processing 10PB of X-ray images.

Resources: Amazon S3 documentation (<https://docs.aws.amazon.com/s3/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>). **Self-check:**

- Which storage class suits data accessed once per quarter but needed within milliseconds?
- What does a delete marker do in a versioned bucket, and how do you undo it?
- When would you choose the Snow Family over DataSync for a migration?
- How does Object Lock protect backups against ransomware?
- Why buffer S3 event notifications through SQS instead of invoking Lambda directly?

Production Note: Practice versioned-object recovery before you need it: with bucket versioning enabled, a “delete” only adds a delete marker — list prior versions with `aws s3api list-object-versions` and restore by removing the marker or copying a previous version. Pair versioning with Object Lock in compliance mode so an attacker with stolen credentials cannot permanently destroy backups.

Week 2: Relational Databases (RDS & Aurora)

Outcomes: distinguish Multi-AZ from read replicas, deploy and fail over Aurora Serverless v2, plan an Oracle-to-Aurora migration with DMS/SCT, and execute a snapshot-restore DR drill with row-count verification.

- **Monday:** Amazon RDS Architecture, Multi-AZ deployments, and Read Replicas.
- **Tuesday:** Amazon Aurora: Cloud-native storage, Global Databases, and Aurora Serverless v2.
- **Wednesday:** Database Migration Service (AWS DMS) and Schema Conversion Tool (SCT).
- **Thursday: Hands-on Project:** Deploy an Amazon Aurora Serverless v2 cluster, create a cross-region read replica, and simulate a regional failover. DR drill: take a manual cluster snapshot, restore it to a *new* cluster, verify row counts against the source with `COUNT(*)` on every table, then delete the restored cluster to stop charges.
- **Friday: Use Case & Architecture:** Architect a zero-downtime migration strategy from a monolithic on-premise Oracle ERP system to Amazon Aurora PostgreSQL.

Resources: Amazon RDS & Aurora documentation (<https://docs.aws.amazon.com/rds/>); AWS DMS documentation (<https://docs.aws.amazon.com/dms/>). **Self-check:**

- How does Multi-AZ differ from a Read Replica in purpose and replication mode?
- How does Aurora’s storage layer differ from a standard RDS volume?
- What does the Schema Conversion Tool do that DMS alone cannot?
- In a restore drill, why restore a snapshot to a new cluster instead of in place?
- Which two values quantify your RPO and RTO during a failover drill?

Production Note: Snapshot restores always create a new cluster — they are never in-place — so a restore drill is the only safe way to prove backups work. Run the drill quarterly, record measured RTO/RPO, and verify data with row counts (or checksums) on every table before deleting the restored cluster. An untested backup is a hypothesis, not a recovery plan.

Week 3: High-Scale NoSQL (Amazon DynamoDB)

Outcomes: model single-table DynamoDB schemas, choose between LSI and GSI, configure Streams and TTL, and estimate RCU/WCU capacity for a high-traffic workload.

- **Monday:** DynamoDB core concepts: Partitions, RCU/WCU (Provisioned vs. On-Demand).
- **Tuesday:** Data Modeling: Primary keys, Sort keys, LSI vs. GSI (Global Secondary Indexes).
- **Wednesday:** DynamoDB Streams, TTL (Time to Live), and Global Tables.
- **Thursday: Hands-on Project:** Create a DynamoDB table, design a single-table schema for a forum, write a Python Boto3 script to insert items, and configure a GSI for querying.
- **Friday: Use Case & Architecture:** Design an ingestion and serving layer for a gaming leaderboard supporting 1 million reads/writes per second globally.

Resources: Amazon DynamoDB documentation (<https://docs.aws.amazon.com/dynamodb/>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>). **Self-check:**

- Why does partition-key choice determine hot-partition risk?
- GSI vs. LSI: which can be added after the table is created?
- What happens to a DynamoDB Streams record after 24 hours unconsumed?
- Why do TTL deletions not consume write capacity?
- Why prefer single-table design over one table per entity?

Week 4: Hybrid Ingestion & Caching

Outcomes: choose between ElastiCache engines and DAX, implement cache-aside query acceleration, explain GraphQL ingestion via AppSync, and design a low-latency API for spiky traffic.

- **Monday:** Amazon ElastiCache (Redis/Memcached) for database acceleration.
- **Tuesday:** DynamoDB Accelerator (DAX).
- **Wednesday:** AWS AppSync (GraphQL) and API Gateway data ingestion.
- **Thursday: Hands-on Project:** Deploy a Redis cluster via ElastiCache. Write a Python script to cache query results from RDS to reduce database load.
- **Friday: Use Case & Architecture:** Architect a high-throughput, low-latency data API for a mobile application that experiences extreme traffic spikes during live sports events.

Resources: Amazon ElastiCache documentation (<https://docs.aws.amazon.com/elasticache/>); Amazon API Gateway documentation (<https://docs.aws.amazon.com/apigateway/>). **Self-check:**

- Redis vs. Memcached: which supports persistence, replication, and complex data types?
- When does DAX beat putting ElastiCache in front of DynamoDB?
- What is a cache stampede and how do jittered TTLs prevent it?
- Why does lazy-loading (cache-aside) suit spiky traffic better than write-through?
- What does API Gateway throttling protect downstream?

Acceptance Criteria

- End-to-end integration
- Architecture
- At least 100k rows (S3) with data
- Monitor performance or experience
- Monthly demand
- Security of Aurora

CAPSTONE PROJECT - Milestone 1: Build an E-Commerce Multi-Tier Data Backend. Deploy Aurora Serverless for user profiles/billing, DynamoDB for active shopping carts and clickstream tracking, S3 for product images, and AWS DMS to continuously replicate Aurora data to S3 for analytics.

Milestone 1 Capstone: Reference Solution Sketch

- **Architecture:** Aurora Serverless v2 for profiles/billing, DynamoDB (single-table, GSI) for carts/clickstream, S3 for images, DMS CDC from Aurora to S3 — decouples analytics from the transactional tier.
- **Steps:** provision Aurora, DynamoDB, and S3 with lifecycle rules; load the Olist seed data; create the DMS replication instance and source/target endpoints; start the CDC task; verify with row-count reconciliation queries; wire CloudWatch alarms on task failure and CDCLatencyTarget.
- **Hardest part — DMS task table mappings:**

```
{
  "rules": [{
    "rule-type": "selection",
    "rule-id": "1",
    "object-locator": { "schema-name": "public", "table-name": "%" },
    "rule-action": "include"
  }]
}
```

- **Pitfalls:** the DMS target-endpoint IAM role missing `s3:PutObject` fails the task silently; a hot cart partition key throttles provisioned DynamoDB tables; a forgotten replication instance leaks cost after the demo.

2 Milestone 2: Data Warehousing & Analytics (Month 2)

Objective: Deep dive into Redshift optimization, Serverless analytics, and Data Lakes.

Learning outcomes: load and benchmark data in Redshift Serverless, tune distribution styles and sort keys, catalog S3 data with Glue and query it via Athena and Spectrum, and enforce row/column-level governance with Lake Formation.

Week 5: Amazon Redshift Architecture & Dimensional Modeling

Outcomes: explain Redshift MPP and Serverless architectures, declare fact-table grain and build a star schema, load data with COPY, and implement SCD Type 2 upserts in SQL.

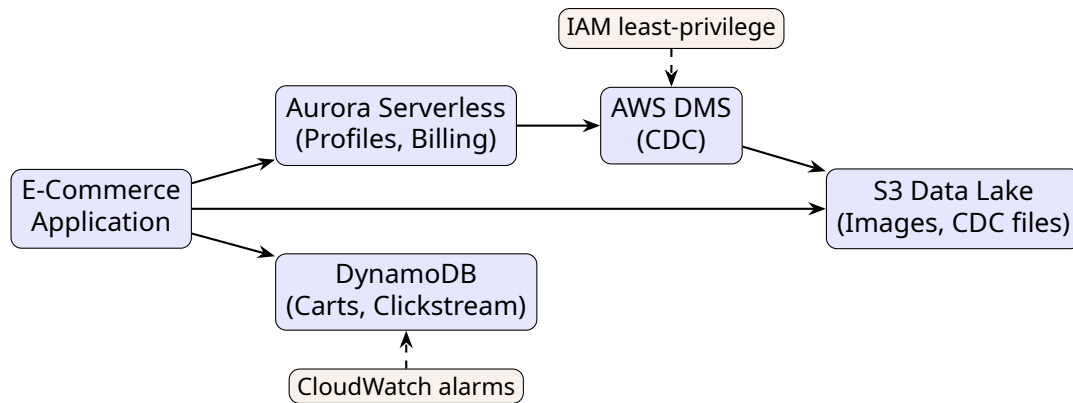


Figure 1: Milestone 1 reference architecture: multi-tier e-commerce data backend

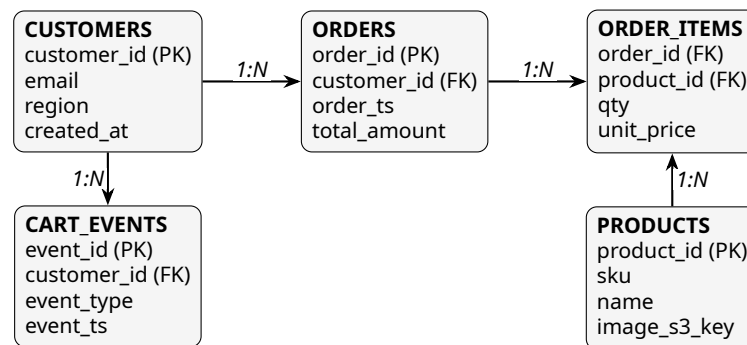


Figure 2: Capstone data model: e-commerce multi-tier backend

- **Monday:** Redshift MPP architecture, Leader vs. Compute nodes, RA3 nodes.
- **Tuesday:** Redshift Serverless vs. Provisioned pricing and scaling.
- **Wednesday:** Data ingestion with COPY commands and Auto-Copy from S3. Kimball dimensional modeling: declare the grain first, then choose the fact type (transaction, periodic snapshot, accumulating snapshot), build conformed dimensions shared across data marts, and decide SCD Type 1 (overwrite) vs. Type 2 (versioned history) per attribute.
- **Thursday: Hands-on Project:** Provision a Redshift Serverless endpoint. Load 10 million rows of synthetic JSON data from S3 using the COPY command with IAM roles. Then declare the grain of a fact_sales table and load conformed dim_customer, dim_product, and dim_date dimensions around it.

```

-- SCD Type 2 upsert in Redshift: close changed rows, then insert new versions
MERGE INTO dim_customer tgt
USING stg_customer src
  ON tgt.customer_id = src.customer_id AND tgt.is_current = TRUE
WHEN MATCHED AND (tgt.segment <> src.segment OR tgt.city <> src.city)
THEN UPDATE SET valid_to = CURRENT_DATE, is_current = FALSE
WHEN NOT MATCHED
THEN INSERT (customer_id, segment, city, valid_from, valid_to, is_current)
  VALUES (src.customer_id, src.segment, src.city,
    CURRENT_DATE, DATE '9999-12-31', TRUE);

-- second pass: insert the new current versions of the rows just closed
INSERT INTO dim_customer (customer_id, segment, city, valid_from, valid_to, is_current)
SELECT src.customer_id, src.segment, src.city,
  CURRENT_DATE, DATE '9999-12-31', TRUE

```

```
FROM stg_customer src
JOIN dim_customer tgt
  ON tgt.customer_id = src.customer_id
  AND tgt.is_current = FALSE
  AND tgt.valid_to = CURRENT_DATE;
```

- **Friday: Use Case & Architecture:** Estimate the monthly cost and recommend an architecture (RA3 vs. Serverless) for a BI workload that is heavily utilized only during end-of-month reporting.

Resources: Amazon Redshift documentation (<https://docs.aws.amazon.com/redshift/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>). **Self-check:**

- Why must you declare the grain before adding columns to a fact table?
- How do periodic snapshot and accumulating snapshot facts differ?
- When is SCD Type 2 required instead of Type 1?
- COPY vs. Auto-Copy: which suits continuous micro-batch loads from S3?
- When does Redshift Serverless beat provisioned RA3 on cost?

Option	Billing model	Best for	
Redshift Serverless	RPU-hours, billed per second	Spiky or intermittent BI workloads, fast on-boarding	
Redshift RA3 (provisioned)	Node-hours plus managed storage per TB-month	Steady, predictable 24/7 analytics at scale	
Athena	Per TB scanned (\$5/TB)	Ad-hoc SQL on S3 with zero infrastructure	
Redshift Spectrum	Per TB scanned, plus a running Redshift cluster	Joining S3 lake data with Redshift tables in one query	

Week 6: Redshift Optimization & Performance

Outcomes: choose distribution styles and sort keys for join-heavy workloads, configure WLM and concurrency scaling, benchmark table designs empirically, and diagnose CPU bottlenecks in daily aggregations.

- **Monday:** Distribution Styles (EVEN, KEY, ALL, AUTO).
- **Tuesday:** Sort Keys (Compound vs. Interleaved) and Compression Encodings.
- **Wednesday:** Workload Management (WLM), Concurrency Scaling, and Materialized Views.
- **Thursday: Hands-on Project:** Create three identical tables in Redshift using different Distribution Styles. Benchmark the execution time of a complex JOIN query on each table.
- **Friday: Use Case & Architecture:** Re-architect a poorly performing multi-petabyte analytics table that is causing CPU bottlenecks during complex daily dashboard aggregations.

Resources: Redshift performance tuning guide (<https://docs.aws.amazon.com/redshift/latest/dg/c-optimizing-query-performance.html>); Data Engineer Handbook (<https://github.com/DataExpert-i/data-engineer-handbook>). **Self-check:**

- Which distribution style avoids data movement when joining two large tables?
- Compound vs. interleaved sort keys: which suits varied filter-column combinations?
- What problem does concurrency scaling solve for BI dashboards?
- When does a materialized view beat a plain view?
- What does rising queue wait time in WLM tell you to change?

Week 7: Data Lakehouse & Amazon Athena

Outcomes: query S3 data with Athena and Spectrum, catalog data with Glue crawlers, convert raw files to Parquet for scan-cost reduction, and design a hybrid lakehouse architecture.

- **Monday:** Amazon Athena: Serverless Presto querying directly on S3.
- **Tuesday:** AWS Glue Data Catalog: Crawlers, Databases, and Tables.
- **Wednesday:** Redshift Spectrum: Querying S3 data directly from Redshift.
- **Thursday: Hands-on Project:** Set up a Glue Crawler to catalog an S3 bucket of raw CSVs. Write an Athena query to aggregate the data, then query the same catalog using Redshift Spectrum.
- **Friday: Use Case & Architecture:** Design a cost-effective "Lakehouse" architecture for a media company wanting to store 50PB of logs in S3 while querying them alongside core business data in Redshift.

Resources: Amazon Athena documentation (<https://docs.aws.amazon.com/athena/>); Apache Iceberg (<https://github.com/apache/iceberg>). **Self-check:**

- Why does Parquet lower Athena cost compared with CSV?
- What does a Glue crawler infer, and when does it infer wrong?
- When is Redshift Spectrum preferable to Athena for the same S3 data?
- How does partition pruning cut bytes scanned?
- What makes Apache Iceberg a lakehouse format beyond plain Parquet files?

Week 8: Data Sharing & Lake Formation

Outcomes: configure Lake Formation row/column-level security, apply LF-Tag-based access control, share data across accounts with Redshift data sharing, and design a data clean room.

- **Monday:** AWS Lake Formation: Centralized data lake governance.
- **Tuesday:** Row-level and column-level security in Lake Formation.
- **Wednesday:** Redshift Data Sharing across AWS accounts.
- **Thursday: Hands-on Project:** Configure AWS Lake Formation over an S3 bucket. Create a data permission policy that restricts a "marketing" IAM role from seeing the "PII_Email" column in Athena.

- **Friday: Use Case & Architecture:** Architect a Data Clean Room for an ad agency to merge their CRM data with third-party advertiser data without exposing raw PII to either party.

Resources: AWS Lake Formation documentation (<https://docs.aws.amazon.com/lake-formation/>); AWS Clean Rooms documentation (<https://docs.aws.amazon.com/clean-rooms/>). **Self-check:**

- What does the default `IAMAllowedPrincipals` grant do, and why revoke it?
- How do LF-Tags differ from named-resource grants?
- What does column-level security protect that table grants cannot?
- How does Redshift data sharing avoid copying data between accounts?
- Why is a clean room safer than exchanging raw CSV exports?

Production Note: *Apply least-privilege in Lake Formation from day one: revoke the default `IAMAllowedPrincipals` super-group on new tables, grant permissions on named columns instead of *, and audit grants with CloudTrail Lake Formation data-plane events. A single overly broad `GRANT` silently defeats every column mask downstream.*

CAPSTONE PROJECT - Milestone 2: Enterprise Data Lakehouse Build. Ingest 5 years of retail data into S3 using Parquet. Catalog it with AWS Glue. Use Lake Formation to apply row/column level security policies based on department. Create Redshift Materialized Views to serve executive dashboards via Redshift Spectrum.

Acceptance Criteria

- End-to-end data pipeline from S3 to Redshift Spectrum
- Architecture diagram showing data flow and security controls
- At least one Parquet table with row-level security
- Monitor data ingestion and export metrics
- Monthly data refresh cycle with 2 optimization steps
- Security audit log showing all grants and revocations

Milestone 2 Capstone: Reference Solution Sketch

- **Architecture:** S3 Parquet lake in raw/curated zones, Glue Crawlers + Data Catalog, Lake Formation LF-Tags for row/column security, Athena for ad-hoc SQL, Redshift Spectrum + materialized views for dashboards.
- **Steps:** land 5 years of retail data in the raw zone; convert to partitioned Parquet (`year=/month=`); run the crawler against the curated zone; revoke `IAMAllowedPrincipals` and grant LF-Tag-based permissions with PII column masks; create the Redshift external schema; build MVs for the executive dashboards.
- **Hardest part — external schema and MV:**

```
CREATE EXTERNAL SCHEMA lakehouse FROM DATA CATALOG
    DATABASE 'retail_lake'
    IAM_ROLE 'arn:aws:iam::123456789012:role/SpectrumRole';
```

```
CREATE MATERIALIZED VIEW mv_monthly_sales AS
SELECT d.year, d.month, p.category, SUM(f.amount) AS revenue
FROM lakehouse.fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
GROUP BY 1, 2, 3;
```

- **Pitfalls:** unpartitioned CSV scans explode Athena cost; crawlers merge schemas when raw files are inconsistent; Spectrum requires the Glue catalog and Redshift in the same region.

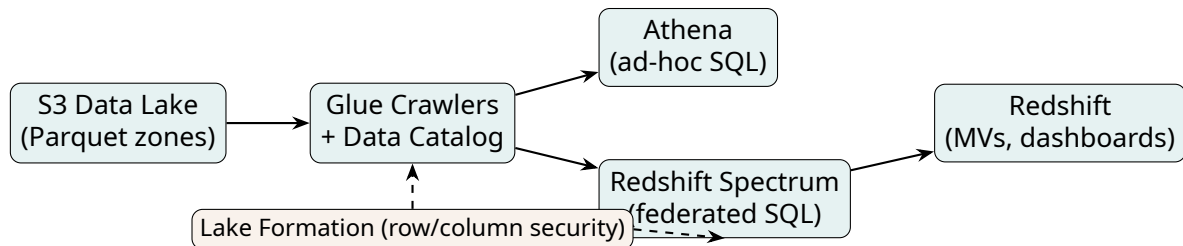


Figure 3: Milestone 2 reference architecture: governed enterprise lakehouse

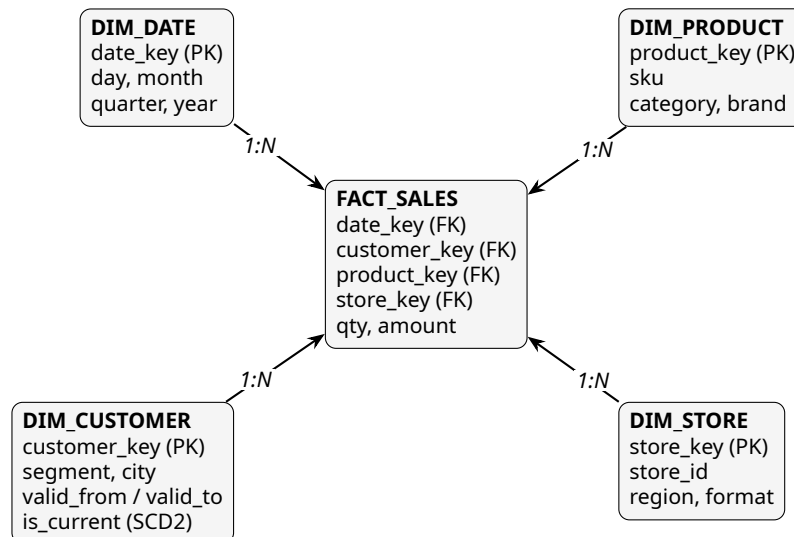


Figure 4: Capstone data model: retail lakehouse star schema

3 Milestone 3: Batch Processing & ETL Orchestration (Month 3)

Objective: Master Spark on EMR, Glue ETL, and Airflow orchestration.

Learning outcomes: submit and tune PySpark jobs on EMR, build serverless Glue ETL pipelines, configure DMS CDC replication to S3, and author production-grade Airflow DAGs on MWAA with failure alerting.

Week 9: Amazon EMR (Managed Hadoop/Spark)

Outcomes: deploy EMR clusters with appropriate node types and Spot capacity, submit PySpark jobs against S3 via EMRFS, compare EMR deployment modes, and plan a Hadoop-to-EMR migration.

- **Monday:** EMR Architecture, Master/Core/Task nodes, and Spot Instances.
- **Tuesday:** EMR Serverless vs. EMR on EC2 vs. EMR on EKS.
- **Wednesday:** EMRFS: Using S3 as the HDFS storage layer.
- **Thursday: Hands-on Project:** Deploy an EMR cluster with Spark. Submit a PySpark job that reads data from S3, performs a distributed aggregation, and writes the output back to S3 in Parquet format.
- **Friday: Use Case & Architecture:** Design a migration strategy for a 100-node on-premise Cloudera Hadoop cluster, prioritizing cost reduction and decoupling compute from storage in AWS.

Resources: Amazon EMR documentation (<https://docs.aws.amazon.com/emr/>); Apache Spark (<https://github.com/apache/spark>). **Self-check:**

- Core vs. Task nodes: which tier can safely run on Spot Instances?
- How does EMRFS decouple storage from compute?
- When would you pick EMR on EKS over EMR on EC2?
- What are bootstrap actions used for?
- Why prefer transient clusters for recurring batch jobs?

Week 10: Serverless ETL (AWS Glue)

Outcomes: build Glue ETL jobs with DynamicFrames and Glue Studio, tune DPUs and Flex execution, enforce idempotent re-runs with job bookmarks, and evaluate Glue vs. EMR fit.

- **Monday:** AWS Glue ETL: DynamicFrames vs. Spark DataFrames.
- **Tuesday:** Glue Studio (Visual ETL) and AWS Glue DataBrew (Data preparation).
- **Wednesday:** Bookmarks, Job tuning (DPUs), and Flex execution.
- **Thursday: Hands-on Project:** Build a visual ETL pipeline in Glue Studio that joins two tables, drops null values, transforms a timestamp, and loads the clean data into Redshift.
- **Friday: Use Case & Architecture:** Evaluate AWS Glue vs. Amazon EMR for a team of business analysts who need to build daily reporting pipelines without managing infrastructure.

Resources: AWS Glue documentation (<https://docs.aws.amazon.com/glue/>); Apache Spark (<https://github.com/apache/spark>). **Self-check:**

- What does a DynamicFrame offer that a Spark DataFrame does not?
- What do job bookmarks guarantee on a re-run?
- When is Glue Flex execution the cheaper choice?
- Why is Glue usually a better fit than EMR for analyst-built pipelines?
- What compute does one DPU represent for billing?

Production Note: Make every Glue job idempotent: enable job bookmarks so retries reprocess only new partitions, write with overwrite-mode partitioned inserts rather than appends, and give jobs a deterministic run date parameter. A re-run after a mid-job failure must never create duplicate rows in Redshift.

Week 11: Change Data Capture (AWS DMS & EventBridge) & Data Contracts

Outcomes: configure DMS CDC replication to S3, define data contracts with the Glue Schema Registry, distinguish breaking from non-breaking schema changes, and monitor CDC lag metrics.

- **Monday:** AWS DMS architecture for continuous replication.
- **Tuesday:** Event-driven data pipelines using Amazon EventBridge.
- **Wednesday:** AWS Lambda for lightweight serverless data transformations. Data contracts with AWS Glue Schema Registry: Avro schemas, compatibility modes (BACKWARD, FORWARD, FULL), contract versioning, breaking vs. non-breaking changes (deleting a field or changing its type breaks consumers; adding an optional field does not), producer-owned contracts with consumer sign-off, and enforcing contracts inside Glue jobs.
- **Thursday: Hands-on Project:** Configure a DMS task with CDC enabled to stream ongoing database changes from an RDS PostgreSQL instance to an S3 Data Lake bucket.
- **Friday: Use Case & Architecture:** Architect a near-real-time synchronization pipeline from an on-premise Oracle database to Amazon Redshift for live BI reporting.

Resources: AWS DMS documentation (<https://docs.aws.amazon.com/dms/>); Amazon EventBridge documentation (<https://docs.aws.amazon.com/eventbridge/>); AWS Glue Schema Registry (<https://docs.aws.amazon.com/glue/latest/dg/schema-registry.html>). **Self-check:**

- Which CloudWatch metrics reveal CDC lag while a task shows “running”?
- BACKWARD vs. FULL compatibility: which permits deleting a field safely?
- Who owns a data contract: producer, consumer, or both?
- Why enforce the schema at ingestion instead of at query time?
- Name one breaking and one non-breaking schema change.

Production Note: Monitor DMS CDC replication lag, not just task status: alarm on the `CDCLatencySource` and `CDCLatencyTarget` CloudWatch metrics, and validate that the source database retains enough transaction log history to survive a replication instance outage. Stale CDC with a “running” task is the classic silent failure.

Week 12: Pipeline Orchestration (Amazon MWAA)

Outcomes: author Airflow DAGs with sensors, operators, and failure alerting, deploy on MWAA, compare with Step Functions orchestration, and design self-healing multi-system pipelines.

- **Monday:** Apache Airflow concepts: DAGs, Operators, Tasks, and XComs.
- **Tuesday:** Amazon MWAA (Managed Workflows for Apache Airflow) architecture.
- **Wednesday:** AWS Step Functions for native AWS serverless orchestration.
- **Thursday: Hands-on Project:** Write a Python DAG for MWAA that: 1. Waits for a file in S3. 2. Triggers an AWS Glue Job. 3. Runs an Athena query. 4. Sends an SNS alert on failure.

```

from datetime import datetime
from airflow import DAG
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
from airflow.providers.amazon.aws.operators.glue import GlueJobOperator
from airflow.providers.amazon.aws.operators.athena import AthenaOperator
from airflow.providers.amazon.aws.operators.sns import SnsPublishOperator

with DAG(
    dag_id="daily_recon_pipeline",
    schedule="@daily",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    default_args={"retries": 2, "retry_delay": 300},
) as dag:
    wait_for_file = S3KeySensor(
        task_id="wait_for_file",
        bucket_name="bank-raw-ingest",
        bucket_key="recon/{{ ds }}/transactions.csv",
        poke_interval=300, timeout=3600,
    )
    run_glue = GlueJobOperator(
        task_id="run_glue", job_name="recon-etl", iam_role_name="GlueExecRole",
    )
    validate = AthenaOperator(
        task_id="validate", query="SELECT COUNT(*) FROM recon.daily_load",
        database="recon", output_location="s3://bank-athena-results/",
    )
    alert = SnsPublishOperator(
        task_id="alert", target_arn="arn:aws:sns:us-east-1:123456789012:data-alerts",
        message="Recon pipeline failed for {{ ds }}", trigger_rule="one_failed",
    )
    wait_for_file >> run_glue >> validate >> alert

```

- **Friday: Use Case & Architecture:** Design a resilient, self-healing orchestration layer for a daily banking reconciliation process spanning 15 dependent systems using MWAA and Step Functions.

Resources: Apache Airflow (<https://github.com/apache/airflow>); Amazon MWAA documentation (<https://docs.aws.amazon.com/mwaa/>). **Self-check:**

- What does `catchup=False` prevent when you deploy a DAG?
- When are Step Functions a better orchestrator than Airflow?
- What do retries with backoff protect a pipeline against?
- Why tune an `S3KeySensor` `poke_interval`?
- When should data move through S3 instead of XComs?

Production Note: Wire MWAA and Glue failures to CloudWatch alarms that publish to an SNS topic subscribed by the on-call runbook (e.g., via Lambda or Incident Manager). Alarm on *FailedTasks* / DAG duration anomalies in MWAA and on Glue job *ERROR* logs — never rely on someone noticing a red box in the Airflow UI. Every alert must link to a runbook entry with triage steps.

Acceptance

- End-to-end pipeline for a file, p
- Architecture: EventBridge, D
- At least one state machine
- Monitoring: CloudWatch, Function
- Monthly: (reports, actions) v
- Security: IAM, hardco

CAPSTONE PROJECT - Milestone 3: Automated Batch ELT Pipeline. Use EventBridge to trigger an AWS Step Function state machine upon file upload to S3. The state machine triggers a Glue DataBrew data quality check, runs an EMR Serverless Spark job to calculate complex ML features, and loads the output into Redshift.

Milestone 3 Capstone: Reference Solution Sketch

- **Architecture:** EventBridge rule on S3 upload triggers a Step Functions state machine: DataBrew DQ check gates an EMR Serverless Spark feature job that loads Redshift; MWAA handles cross-day scheduling dependencies.
- **Steps:** enable S3 event notifications to EventBridge; define the state machine with a Choice state on the DQ result; create the EMR Serverless application and Spark job; COPY the curated output into Redshift; publish failures to SNS.
- **Hardest part — the DQ gate Choice state (ASL):**

```
"CheckDataQuality": {
  "Type": "Choice",
  "Choices": [{
    "Variable": "$dq_status",
    "StringEquals": "FAILED",
    "Next": "AlertAndHalt"
  }],
  "Default": "RunSparkFeatures"
}
```

- **Pitfalls:** Step Functions has a 256 KB payload limit — pass S3 keys, never data; DataBrew profile jobs scan full datasets and surprise on cost; missing bucket event configuration means the pipeline never fires.

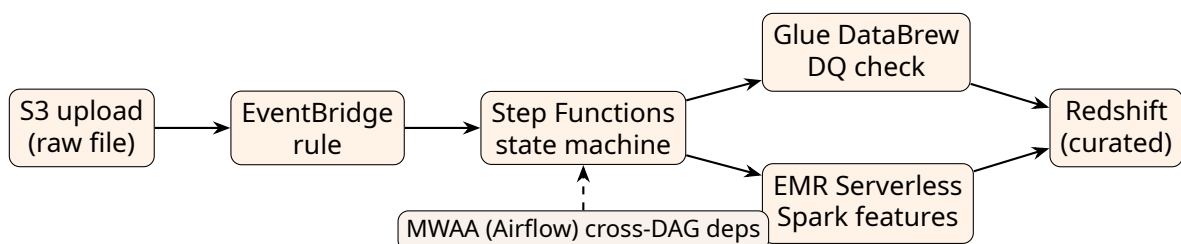


Figure 5: Milestone 3 reference architecture: automated batch ELT pipeline

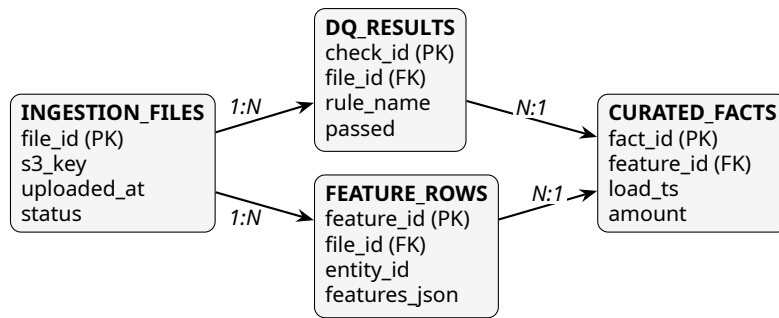


Figure 6: Capstone data model: automated batch ELT pipeline

4 Milestone 4: Streaming & Real-Time Analytics (Month 4)

Objective: Conquer unbounded data, Kinesis, Kafka, and real-time visualization.

Learning outcomes: design and scale Kinesis Data Streams, build producers and consumers with Boto3, configure Firehose delivery with inline transformation, implement Flink windowed aggregations, and publish real-time QuickSight dashboards.

Week 13: Message Queuing & Streaming Ingestion

Outcomes: size Kinesis streams from shard throughput limits, write retrying Boto3 producers with per-key ordering, build idempotent consumers, and compare Kinesis with MSK.

- **Monday:** Decoupling with Amazon SQS and SNS (Fan-out pattern).
- **Tuesday:** Amazon Kinesis Data Streams (KDS): Shards, Partition Keys, and scaling.
- **Wednesday:** Amazon MSK (Managed Streaming for Apache Kafka) vs. Kinesis.
- **Thursday: Hands-on Project:** Create a Kinesis Data Stream. Write a Python producer script (Boto3) to stream mock IoT temperature sensor data, and a consumer to print the data.

```

import json, random, time
import boto3

kinesis = boto3.client("kinesis", region_name="us-east-1")
STREAM = "iot-temperature-stream"

def make_reading(sensor_id: int) -> dict:
    return {
        "sensor_id": f"sensor-{sensor_id}",
        "ts": int(time.time()),
        "temp_c": round(random.uniform(15.0, 35.0), 2),
        "battery_pct": random.randint(20, 100),
    }

while True:
    records = []
    for sensor_id in range(1, 51):
        records.append({
            "Data": json.dumps(make_reading(sensor_id)).encode("utf-8"),
            "PartitionKey": f"sensor-{sensor_id}", # per-sensor ordering
        })
    resp = kinesis.put_records(StreamName=STREAM, Records=records)
    if resp["FailedRecordCount"]:
        # retry only failed records with backoff (ProvisionedThroughputExceeded)

```



```

failed = [r for r, o in zip(records, resp["Records"]) if "ErrorCode" in o]
time.sleep(0.5)
kinesis.put_records(StreamName=STREAM, Records=failed)
time.sleep(1)

```

- **Friday: Use Case & Architecture:** Architect a global telemetry ingestion point for 5 million mobile games, ensuring exactly-once processing semantics and strict message ordering.

Resources: Amazon Kinesis documentation (<https://docs.aws.amazon.com/kinesis/>); Apache Kafka (<https://github.com/apache/kafka>). **Self-check:**

- Which two throughput limits determine how many shards a stream needs?
- Why does partition-key choice control per-entity ordering?
- What does a rising `IteratorAgeMilliseconds` warn about?
- How do you build exactly-once effects on at-least-once delivery?
- Kinesis vs. MSK: which is billed per shard-hour and record volume?

Production Note: Design consumers to be idempotent — Kinesis delivers at-least-once, so deduplicate on a record ID (e.g., in Redis or a DynamoDB conditional put) before writing downstream. Route poison records that fail repeatedly to a dead-letter location (SQS DLQ or an S3 error prefix) instead of blocking the shard iterator, and alarm on `IteratorAgeMilliseconds` to catch slow consumers before the 24-hour retention window expires.

Week 14: Real-Time Delivery (Kinesis Firehose)

Outcomes: configure Firehose delivery with inline Lambda transformation, convert JSON to Parquet in flight, enable source-record backup, and design real-time anonymization before delivery.

- **Monday:** Amazon Kinesis Data Firehose architecture and destinations.
- **Tuesday:** Inline data transformation using AWS Lambda in Firehose.
- **Wednesday:** Format conversion (JSON to Parquet/ORC) on the fly.
- **Thursday: Hands-on Project:** Connect a Kinesis Firehose delivery stream to your KDS. Configure a Lambda function to uppercase all incoming strings, convert the data to Parquet, and output to S3.
- **Friday: Use Case & Architecture:** Design a pipeline to anonymize incoming healthcare HL7 messages in real-time before streaming them into a Redshift cluster for analysis.

Resources: Kinesis Data Firehose documentation (<https://docs.aws.amazon.com/firehose/>); AWS Lambda documentation (<https://docs.aws.amazon.com/lambda/>). **Self-check:**

- Name three native Firehose destinations.
- Why enable source-record backup to S3 on a delivery stream?
- What do inline Lambda transformations add to latency and cost?
- Why convert JSON to Parquet during Firehose delivery?
- Where does data land when Redshift delivery keeps failing?

Production Note: Always enable Firehose source-record backup to an S3 raw prefix and configure retry duration for Lambda transformations. For Redshift destinations, alarm on `DeliveryToRedshift.Success` and keep the intermediate S3 bucket lifecycle-tiered — failed deliveries silently accumulate there otherwise.

Week 15: Stream Processing (Apache Flink on AWS)

Outcomes: explain event time vs. processing time, apply watermarks for late events, implement tumbling/sliding/session windows and stream-stream joins in Flink SQL, manage keyed state with the RocksDB state backend and state TTL, distinguish checkpoints from savepoints, and deliver end-to-end exactly-once with transactional or idempotent sinks.

- **Monday:** Managed Service for Apache Flink (formerly Kinesis Data Analytics).
- **Tuesday:** Event Time vs. Processing Time and Watermarks in Flink.
- **Wednesday:** Windowing strategies: Tumbling, Sliding, and Session windows. Stream-stream joins: interval and window joins bounded by watermarks, and their state retention implications. State management deep dive: keyed state, the RocksDB state backend, checkpointing vs. savepoints, state TTL, and why unbounded state kills streaming jobs.
- **Thursday: Hands-on Project:** Write a Flink SQL application that reads from a Kinesis Stream, groups user clicks into 5-minute Tumbling Windows, counts the clicks, and writes the output to a new stream.
- **Friday:** End-to-end exactly-once delivery: checkpointing plus idempotent or transactional sinks (two-phase commit to Kafka/S3) vs. at-least-once with downstream dedup. **Use Case & Architecture:** Troubleshoot a scenario where a streaming dashboard is showing inaccurate daily aggregations due to mobile devices going offline and uploading data late.

```
-- Watermarks declared in DDL on both input streams
CREATE TABLE impressions (
  ad_id STRING, imp_ts TIMESTAMP(3),
  WATERMARK FOR imp_ts AS imp_ts - INTERVAL '10' SECOND
) WITH ('connector' = 'kinesis', 'stream' = 'imps', ...);

CREATE TABLE clicks (
  user_id STRING, ad_id STRING, click_ts TIMESTAMP(3),
  WATERMARK FOR click_ts AS click_ts - INTERVAL '5' SECOND
) WITH ('connector' = 'kinesis', 'stream' = 'clicks', ...);

-- Interval join: attribute clicks to impressions seen up to 15 min prior.
-- Watermarks let Flink drop state older than the join interval.
SELECT i.ad_id, c.user_id, c.click_ts
FROM impressions i
JOIN clicks c ON i.ad_id = c.ad_id
WHERE c.click_ts BETWEEN i.imp_ts AND i.imp_ts + INTERVAL '15' MINUTE;
```

Resources: Amazon Managed Service for Apache Flink documentation (<https://docs.aws.amazon.com/managed-flink/>); Apache Flink documentation (<https://nightlies.apache.org/flink/flink-docs-st>)

Self-check:

- Why does unbounded keyed state eventually kill a streaming job?
- Checkpoint vs. savepoint: which do you trigger before a redeployment, and why?
- In a stream-stream interval join, what bounds how long join state is retained?

- Which two components combine for end-to-end exactly-once delivery to Kafka or S3?
- When is at-least-once plus downstream dedup preferred over exactly-once?

Production Note: *Enforce savepoint discipline on every deployment: stop the job with a savepoint, never with a plain cancel, and pin the application to restore from that savepoint on the next start. Checkpoint state cannot survive code changes that alter the operator graph, and Managed Service for Apache Flink snapshots are not portable across incompatible changes — a missed savepoint before redeploy means reprocessing or lost state.*

Week 16: Business Intelligence (Amazon QuickSight) & Reverse ETL

Outcomes: build QuickSight dashboards on Redshift and Athena, enforce row-level security on shared datasets, implement reverse ETL via UNLOAD to a CRM API, and reason about sync-frequency trade-offs.

- **Monday:** Amazon QuickSight Architecture and SPICE engine.
- **Tuesday:** Connecting QuickSight to Redshift, Athena, and S3.
- **Wednesday:** Row-level security (RLS) and ML Insights in QuickSight. Reverse ETL: Redshift UNLOAD of curated segments/audiences to S3, a Lambda function pushing them to a CRM/marketing API, identity resolution (matching the same person across systems before sync), and sync-frequency trade-offs (hourly freshness vs. API rate limits and cost).
- **Thursday: Hands-on Project:** Connect Amazon QuickSight to your Redshift cluster. Create a real-time dashboard monitoring sales, apply a filter, and set up an ML Anomaly Detection alert.
- **Friday: Use Case & Architecture:** Design the BI access layer for an enterprise where 500 analysts need to query centralized data, ensuring regional sales managers can only see data for their specific regions (RLS).

Resources: Amazon QuickSight documentation (<https://docs.aws.amazon.com/quicksight/>); Redshift UNLOAD documentation (https://docs.aws.amazon.com/redshift/latest/dg/r_UNLOAD.html); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>).

Self-check:

- What does SPICE accelerate, and what is its capacity limit?
- How does QuickSight row-level security filter a shared dataset?
- What is reverse ETL, and which operational systems does it feed?
- Why must identity resolution happen before syncing audiences to a CRM?
- Hourly vs. daily sync: which trade-offs decide the frequency?

Acceptance Criteria

- End-to-end flow from data source to dashboard
- Architecture diagram showing data flow from Kinesis to Redshift to QuickSight
- At least one alert routing rule in Redshift
- Monitoring dashboard in QuickSight showing fraud alerts
- Monthly cost optimization report
- Security audit report showing no vulnerabilities

CAPSTONE PROJECT - Milestone 4: Real-Time Fraud Detection. Build a Python script simulating credit card swipes sent to Kinesis Data Streams. Create a Managed Flink application that uses a Sliding Window to detect if a card is used in two different cities within 10 minutes, flags it, routes it via Firehose to Redshift, and displays the live fraud alerts in QuickSight.

Milestone 4 Capstone: Reference Solution Sketch

- **Architecture:** Kinesis Data Streams ingest → Managed Flink sliding-window detection on event time → Firehose (S3 raw backup + Redshift alert delivery) → QuickSight live dashboard.
- **Steps:** create the stream with on-demand capacity; write the swipe producer with `card_id` as partition key; build the Flink SQL app detecting cross-city use; route flagged alerts through Firehose to Redshift; build the QuickSight dashboard; deduplicate alert writes on `txn_id`.
- **Hardest part — the two-city sliding-window query:**

```
SELECT card_id, window_start, window_end,
       COUNT(DISTINCT city) AS city_count
FROM TABLE(HOP(TABLE swipes, DESCRIPTOR(event_ts),
                 INTERVAL '10' MINUTES, INTERVAL '1' MINUTES))
GROUP BY card_id, window_start, window_end
HAVING COUNT(DISTINCT city) > 1;
```

- **Pitfalls:** processing-time windows flag false positives on late mobile events; under-provisioned Flink KPIs cause backpressure that ages the shard iterator; the raw card PAN must be tokenized before anything lands in Redshift.

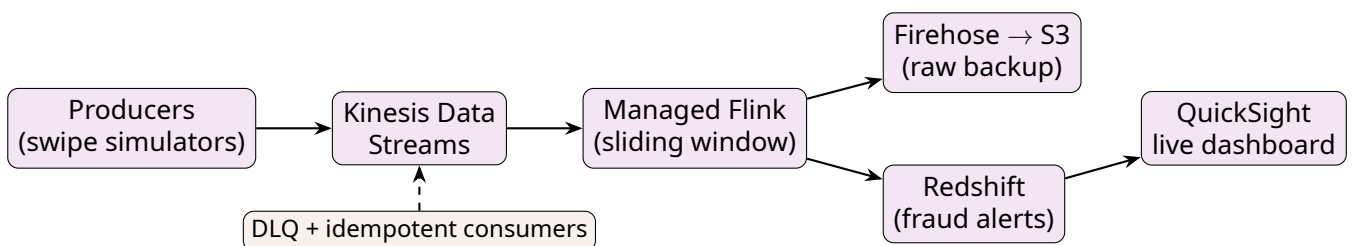


Figure 7: Milestone 4 reference architecture: real-time fraud detection stream

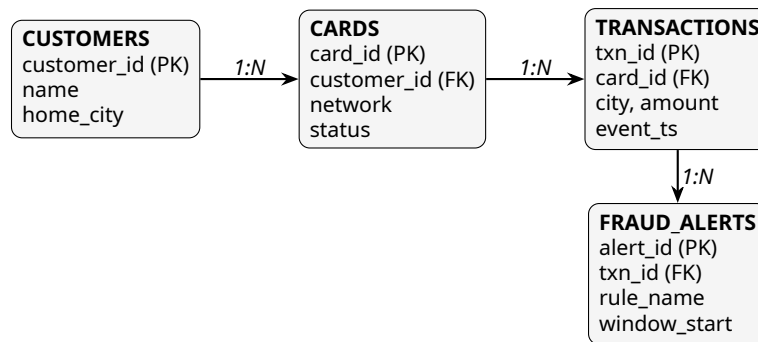


Figure 8: Capstone data model: real-time fraud detection

5 Milestone 5: Machine Learning Operationalization (Month 5)

Objective: Embed AI into data pipelines and manage the MLOps lifecycle with SageMaker.

Learning outcomes: train in-database models with Redshift ML, prepare data and deploy end-points in SageMaker, build Feature-Store-backed SageMaker Pipelines with Model Monitor drift detection, and integrate managed AI APIs into Lambda/Glue pipelines.

Week 17: In-Database Machine Learning (Redshift ML)

Outcomes: train Redshift ML models via SQL, select problem types and objectives, score live data with model functions, and judge when to graduate to full SageMaker.

- **Monday:** Redshift ML architecture (integration with SageMaker Autopilot).
- **Tuesday:** Creating models: Regression, Classification, and K-Means.
- **Wednesday:** Model evaluation and scoring directly via SQL queries.
- **Thursday: Hands-on Project:** Use Redshift ML to train a logistic regression model on sample customer data to predict churn. Run a SQL query using the new model function to predict churn on live data.

```

CREATE MODEL customer_churn_model
FROM (
    SELECT tenure_months, monthly_spend, support_tickets,
           avg_session_min, is_churned
    FROM analytics.customers
)
TARGET is_churned
FUNCTION fn_predict_churn
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftMLRole'
SETTINGS (
    S3_BUCKET 'ml-artifacts-bucket',
    PROBLEM_TYPE 'binary_classification',
    OBJECTIVE 'F1'
);

-- score live customers once the model state is READY
SELECT customer_id, churn_prob
FROM (
    SELECT customer_id,
           fn_predict_churn(tenure_months, monthly_spend,
                           support_tickets, avg_session_min) AS churn_prob
    FROM analytics.customers_current
)
  
```

```
WHERE churn_prob > 0.7;
```

- **Friday: Use Case & Architecture:** Architect a system to automatically forecast daily inventory needs for 1,000 retail stores using purely SQL-based pipelines to minimize data movement.

Resources: Redshift ML documentation (<https://docs.aws.amazon.com/redshift/latest/dg/machine-learning.html>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-h>)
Self-check:

- What part of training does Redshift ML delegate to SageMaker?
- Why is in-database ML attractive for SQL-only forecasting pipelines?
- What does the `FUNCTION` clause of `CREATE MODEL` define?
- Why choose F1 over accuracy for imbalanced churn data?
- When should you graduate from Redshift ML to full SageMaker?

Week 18: Amazon SageMaker Foundations

Outcomes: prepare data with Data Wrangler; train and deploy XGBoost via Autopilot, choose between real-time/serverless/batch endpoints, and secure training-data access through Lake Formation.

- **Monday:** SageMaker Studio, Notebook Instances, and SageMaker Data Wrangler.
- **Tuesday:** Built-in Algorithms (XGBoost, BlazingText) vs. Custom Containers.
- **Wednesday:** SageMaker Endpoints: Real-time, Serverless, and Batch Transform.
- **Thursday: Hands-on Project:** Use SageMaker Data Wrangler to clean an S3 dataset. Train an XGBoost model using SageMaker Autopilot. Deploy the model to a serverless endpoint and send a test payload.
- **Friday: Use Case & Architecture:** Design a workflow for data scientists to train custom PyTorch models on GPU instances while securely accessing training data stored in a governed S3 data lake.

Resources: Amazon SageMaker documentation (<https://docs.aws.amazon.com/sagemaker/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>).
Self-check:

- Real-time vs. serverless vs. batch transform: which fits sporadic traffic?
- What does SageMaker Data Wrangler export after interactive cleaning?
- When do you need a custom container instead of built-in XGBoost?
- How does Autopilot pick its final model?
- Why route training-data access through Lake Formation instead of direct S3?

Week 19: MLOps & SageMaker Pipelines

Outcomes: build SageMaker Pipelines with Feature Store integration, gate deployments through the Model Registry, configure Model Monitor drift detection, and design a human-approved re-training loop.

- **Monday:** SageMaker Feature Store: Online vs. Offline serving.
- **Tuesday:** SageMaker Pipelines for MLOps orchestration.
- **Wednesday:** SageMaker Model Monitor (Data Drift and Concept Drift).
- **Thursday: Hands-on Project:** Build a simple SageMaker Pipeline (Preprocess → Train → Evaluate → Register). Execute the pipeline from a Jupyter notebook.
- **Friday: Use Case & Architecture:** Architect an MLOps lifecycle that detects data drift in a production recommendation engine and automatically triggers a retraining pipeline using EventBridge.

Resources: SageMaker Pipelines documentation (<https://docs.aws.amazon.com/sagemaker/latest/dg/pipelines.html>); OpenLineage (<https://github.com/OpenLineage/OpenLineage>). **Self-check:**

- Online vs. offline Feature Store: which serves training, which inference?
- What does the Model Registry gate before deployment?
- Data drift vs. concept drift: what is the difference?
- Why gate automatic retraining behind human approval?
- What baseline does Model Monitor need to detect violations?

Production Note: *Treat drift alerts as incidents, not curiosities: Model Monitor violations should publish to SNS with a runbook that defines who acknowledges, what the rollback path is (previous Model Registry version), and when retraining auto-triggers. Gate automatic retraining behind a human approval step in the pipeline — drift can also mean broken upstream ingestion, and retraining on corrupt data makes things worse.*

Week 20: Unstructured Data & AI Services

Outcomes: extract text and metadata with Textract and Rekognition, integrate AI APIs into Lambda/Glue pipelines, index results into OpenSearch, and manage PII risk in OCR output.

- **Monday:** Amazon Rekognition (Images/Video) and Amazon Textract (OCR).
- **Tuesday:** Amazon Comprehend (NLP) and Amazon Translate.
- **Wednesday:** Integrating AWS AI APIs within Lambda and Glue pipelines.
- **Thursday: Hands-on Project:** Write a Lambda function triggered by an S3 upload that sends a scanned PDF document to Amazon Textract, extracts the key-value pairs, and saves them as JSON back to S3.
- **Friday: Use Case & Architecture:** Design a scalable pipeline to process terabytes of uploaded user images, extract text (OCR), tag objects, and make the metadata searchable using OpenSearch.

Resources: Amazon Textract documentation (<https://docs.aws.amazon.com/textract/>); Amazon Rekognition documentation (<https://docs.aws.amazon.com/rekognition/>). **Self-check:**

- Textract vs. Comprehend: which extracts key-value pairs from scanned forms?
- Why trigger AI APIs from S3 events via Lambda?
- When must you switch from synchronous to asynchronous Textract jobs?
- How do you make extracted document metadata searchable?
- What PII risks exist when persisting raw OCR output?

CAPSTONE PROJECT - Milestone 5: End-to-End MLOps Pipeline. Use MWAA (Airflow) to orchestrate a pipeline that extracts features via Athena, pushes them to SageMaker Feature Store, trains a custom model using SageMaker Training Jobs, registers the model in the Model Registry, and sets up SageMaker Model Monitor for serving skew.

Acceptance Criteria

- End-to-end pipeline
- README
- Architecture
- Model Registry
- At least one model
- Monitor for failure
- Monthly retraining
- Security and encryption

Milestone 5 Capstone: Reference Solution Sketch

- **Architecture:** Athena feature extraction → SageMaker Feature Store (offline for training, online for serving) → SageMaker Pipeline (preprocess → train → evaluate → register) → Model Registry approval → endpoint with Model Monitor; MWAA orchestrates; drift violations feed back to retraining via EventBridge.
- **Steps:** create the feature group and ingest features; define pipeline steps with `sagemaker.workflow`; gate registration on an F1 threshold; build the Model Monitor baseline and schedule; wire violations to SNS with a human approval step before retraining.

- **Hardest part — the pipeline quality gate:**

```
cond = ConditionStep(
    name="QualityGate",
    conditions=[ConditionGreaterThanOrEqualTo(
        left=JsonGet(step=eval_step, property_file=metrics,
                      json_path="classification.f1"),
        right=0.80)],
    if_steps=[register_step],
    else_steps=[FailStep(name="RejectModel",
                        error_message="F1 below 0.80")],
)
```

- **Pitfalls:** online/offline feature skew when transforms are coded twice; a Monitor baseline computed from stale training data misses real drift; a forgotten always-on endpoint dwarfs the entire training bill — use serverless endpoints or batch transform.

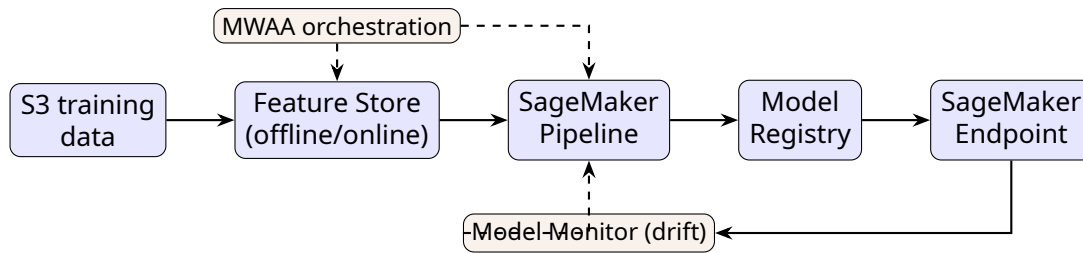


Figure 9: Milestone 5 reference architecture: MLOps lifecycle with drift feedback loop

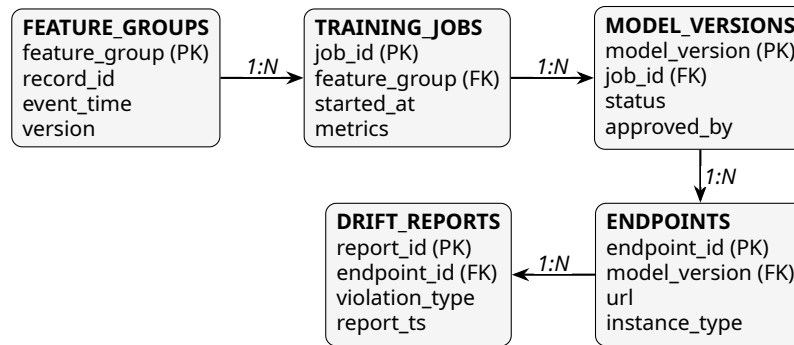


Figure 10: Capstone data model: end-to-end MLOps pipeline

6 Milestone 6: Security, Governance & Exam Prep (Month 6)

Objective: Secure the perimeter, implement Data Mesh, and prepare for certification.

Learning outcomes: enforce least-privilege IAM and private networking for data services, automate data-quality rulesets and lineage, detect and quarantine PII with Macie, provision data infrastructure with Terraform and dbt, and build a CI/CD pipeline for data workloads.

Week 21: Security, IAM & Networking

Outcomes: apply least-privilege IAM with boundaries and SCPs, secure traffic with VPC endpoints and PrivateLink, manage keys and secrets with KMS and Secrets Manager, and design an anti-exfiltration data perimeter.

- **Monday:** AWS IAM Deep Dive: Roles, Policies, Boundaries, and SCPs.
- **Tuesday:** VPC Networking: Endpoints (Gateway vs. Interface) and PrivateLink.
- **Wednesday:** AWS KMS (Key Management Service) and AWS Secrets Manager.
- **Thursday: Hands-on Project:** Create a VPC Endpoint for S3 to ensure traffic doesn't traverse the public internet. Attempt to access the S3 bucket from an EC2 instance without an Internet Gateway.
- **Friday: Use Case & Architecture:** Design a highly secure data perimeter for a health-care provider ensuring no data exfiltration can occur to outside AWS accounts even if IAM credentials are compromised.

Resources: AWS IAM documentation (<https://docs.aws.amazon.com/iam/>); AWS PrivateLink documentation (<https://docs.aws.amazon.com/privatelink/>). **Self-check:**

- Gateway vs. interface VPC endpoint: which is free for S3 traffic?

- What does a permissions boundary limit on a role?
- Why is automatic Secrets Manager rotation better than static keys?
- How do SCPs differ from IAM policies?
- What exfiltration path does PrivateLink close that security groups cannot?

Production Note: *Cost guardrail: interface VPC endpoints are billed per AZ-hour plus per-GB — for high-volume S3 traffic always prefer the free Gateway endpoint, and consolidate interface endpoints (e.g., one shared services VPC) instead of one per workload VPC. Also set up AWS Budgets alerts per account before the security hardening project, so the perimeter itself doesn't produce a surprise bill.*

Week 22: Data Governance & Quality

Outcomes: author Glue Data Quality rulesets with alerting, implement tag-based cross-account sharing, trace failures with lineage, and design a domain-owned data mesh.

- **Monday:** AWS Lake Formation: Cross-account data sharing and Tag-based access control.
- **Tuesday:** AWS Glue Data Quality: Automated rulesets and alerts.
- **Wednesday:** Data Lineage with Amazon SageMaker ML Lineage Tracking.
- **Thursday: Hands-on Project:** Implement an AWS Glue Data Quality ruleset within an ETL job to fail the pipeline if a required column is more than 5% null. Route failure alerts to SNS.

```
Rules = [
    IsComplete "order_id",
    IsUnique "order_id",
    ColumnValues "order_amount" >= 0,
    ColumnLength "email" between 5 and 254,
    Completeness "email" >= 0.95 with threshold 0.9
]

# In the Glue job (PySpark) evaluate and route the outcome:
# from awsglue.data_quality import DataQualityEvaluator # pseudocode shape
# dq_results = EvaluateDataQuality.apply(frame, ruleset)
# if dq_results.status == "FAILED":
#     sns.publish(TopicArn=ALERTS_TOPIC, Message=str(dq_results))
#     raise RuntimeError("DQ gate failed - load halted")
```

- **Friday: Use Case & Architecture:** Architect a Data Mesh for a global conglomerate with 5 independent domains, ensuring centralized governance via Lake Formation but decentralized data engineering.

Resources: AWS Glue Data Quality documentation (<https://docs.aws.amazon.com/glue/latest/dg/glue-data-quality.html>); Great Expectations (https://github.com/great-expectations/great_expectations); DataHub (<https://github.com/datahub-project/datahub>). **Self-check:**

- What happens to the pipeline when a Glue DQ rule fails?
- How do LF-Tags simplify cross-account data sharing?
- Why consult lineage when a data-quality check fails?
- ColumnExists vs. Completeness: which catches schema drift?

- In a data mesh, who owns the quality of a data product?

Production Note: *Handle schema drift explicitly: register schemas in the Glue Schema Registry (or enforce contracts via DQ rules like `ColumnExists`), and decide per pipeline whether new columns are adopted, ignored, or page the on-call. Combine DQ rule results with OpenLineage/DataHub so a quality failure can be traced to the exact upstream table that introduced it.*

Week 23: Compliance & Data Privacy (Amazon Macie)

Outcomes: discover and classify PII with Macie, build auditable quarantine workflows, propagate erasure requests across lake zones, warehouse, streams, and backups, and implement crypto-shredding with per-subject KMS keys.

- **Monday:** Amazon Macie: Automated PII discovery and classification in S3.
- **Tuesday:** Masking, Tokenization, and Crypto-based hashing in pipelines. GDPR “right to erasure” propagation: an erasure request must reach raw S3 lake zones, curated/warehouse tables in Redshift, streaming buffers bounded by Kinesis retention windows (24 hours to 365 days), downstream derived tables and aggregates, caches, and clones/backups including Glacier archives and Redshift snapshots. Practical pattern: crypto-shredding with per-subject KMS keys — encrypt each data subject’s records under a subject-specific KMS data key, then delete that key to render every copy (including backups and cross-region replicas) permanently unrecoverable — versus row-level deletes with downstream replay through CDC. Handle legal-hold and audit-trail exceptions explicitly: an S3 Object Lock legal hold overrides retention and blocks deletion until released, so erasure jobs must check hold status and route held records to an exception queue rather than failing or silently skipping them.
- **Wednesday:** AWS CloudTrail and AWS Config for auditing data access.
- **Thursday: Hands-on Project:** Enable Amazon Macie on an S3 bucket. Upload a CSV containing mock credit card numbers. Review the Macie findings report and trigger a Lambda function to quarantine the file.
- **Friday: Use Case & Architecture:** Design a pipeline that ingests third-party customer data, automatically identifies PII using Macie, tokenizes it using a KMS key in Glue, and stores the safe version in the data warehouse.

Resources: Amazon Macie documentation (<https://docs.aws.amazon.com/macie/>); AWS CloudTrail documentation (<https://docs.aws.amazon.com/cloudtrail/>). **Self-check:**

- What does Macie detect beyond regex-matched patterns?
- Why quarantine suspect files with Object Lock instead of deleting them?
- Tokenization vs. hashing: which transformation is reversible?
- What do CloudTrail data events record for S3?
- Why tune Macie custom data identifiers before rollout?

Production Note: *Quarantine workflows must be non-destructive and auditable: move suspect objects to a quarantine bucket with Object Lock rather than deleting them, and log the Macie finding ID with every quarantine action to CloudTrail. Tune Macie custom data identifiers before rollout — an over-broad regex generates thousands of findings and trains the team to ignore real PII alerts.*

Production Note: A naive `DELETE` does not erase data on AWS: S3 versioning retains prior object versions, Redshift snapshots and automated backups keep deleted rows until they expire, Kinesis records persist for the full retention window, and Glacier archives are immutable until their retention period ends. Erasure runbooks must enumerate every copy location, then either cryptoshred the subject's KMS key (which covers backups too) or schedule versioned-object deletion plus snapshot expiry — and verify afterwards with a Macie re-scan that no residual PII remains.

Week 24: Production DataOps (IaC, dbt & CI/CD) & Certification Prep

Outcomes: provision data infrastructure with Terraform, build tested dbt models on Redshift, implement GitHub Actions CI/CD with environment promotion, and execute zero-downtime expand-contract schema migrations.

- **Monday:** Terraform for AWS data infrastructure: provisioning S3 buckets, Redshift Serverless, Glue jobs, and IAM roles as code; CloudFormation and CDK as alternative IaC approaches.
- **Tuesday:** dbt on Redshift: models, tests, documentation, and managing dependencies with sources and `ref()`.
- **Wednesday:** CI/CD concepts for data pipelines: GitHub Actions workflows and environment promotion from Dev to Prod. Zero-downtime schema evolution with the expand → migrate → contract pattern: first add the new nullable column (expand); then dual-write new writes and backfill history while reads still use the old column (migrate); then switch reads over via a backward-compatible view shim for consumers and, only after all consumers are verified, drop the old column in a later release (contract). Coordinate each phase as its own dbt model change and GitHub Actions deployment so Redshift DDL never breaks running dashboards or Glue jobs mid-release.
- **Thursday: Hands-on Project:** Create a GitHub repository containing a Terraform module that provisions an S3 data lake bucket and a Glue job, plus a dbt project with 3 models and 5 tests against Redshift. Add a GitHub Actions workflow that runs dbt tests on every pull request and deploys on merge to main.

```
resource "aws_s3_bucket" "data_lake" {
  bucket = "acme-data-lake-prod"
  tags   = { Environment = "prod", Owner = "data-platform" }
}

resource "aws_s3_bucket_versioning" "data_lake" {
  bucket = aws_s3_bucket.data_lake.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_lifecycle_configuration" "data_lake" {
  bucket = aws_s3_bucket.data_lake.id

  rule {
    id      = "tier-raw-to-glacier"
    status  = "Enabled"
    filter { prefix = "raw/" }

    transition {
      days          = 30
      storage_class = "GLACIER"
    }
  }

  noncurrent_version_expiration { noncurrent_days = 90 }
}
```

```
}  
}
```

- **Friday:** Compressed certification prep: review the AWS Certified Data Engineer – Associate exam domains, note your weak areas, and schedule both full-length mock exams for the weekend following the program.

Resources: Terraform AWS Provider (<https://registry.terraform.io/providers/hashicorp/aws/latest/docs>); dbt Core (<https://github.com/dbt-labs/dbt-core>); GitHub Actions documentation (<https://docs.github.com/actions>). **Self-check:**

- Why store Terraform state remotely with locking?
- What do dbt tests validate that table schemas alone cannot?
- Why promote dbt runs Dev → Prod instead of sharing one workspace?
- Why prefer OIDC federation over long-lived AWS keys in CI?
- What must the PR-check workflow prove before merge to main?

Production Note: *Keep Terraform state in a remote S3 backend with DynamoDB locking and restrict terraform apply to CI — engineers run plan in pull requests only. Gate the deploy workflow on dbt tests passing in the Dev workspace, and require an approval environment for Prod promotion. Never store AWS credentials in repo secrets as long-lived keys; use OIDC federation from GitHub Actions to an IAM role.*

Production Note: *Deploy order matters more than the migration plan: one team renamed order_amount to total_amount and shipped the dbt model change and the QuickSight dataset change in a single GitHub Actions release — the dbt migration applied minutes before the dashboard refresh, and every executive dashboard errored for an hour because nothing pointed at the old column during the gap. The fix that stuck: contract-phase drops go in a separate pull request, merged no sooner than one full deploy cycle after all consumers are verified on the new schema, with the compatibility view shim kept until then.*

CAPSTONE PROJECT - Milestone 6: Secure, Governed Data Mesh. Build a multi-account architecture using AWS Organizations. Account A ingests streaming PII data and uses Macie/Lambda to tokenize it. Account B (Analytics) hosts Redshift. Account C (Governance) uses Lake Formation to enforce cross-account tag-based security and VPC Endpoints to lock down all traffic.

Acceptance

- End-to-end repo R
- Architecture ernanc
- At least Macie
- Monitoring tions to
- Monthly with 2
- Security Format

Milestone 6 Capstone: Reference Solution Sketch

- **Architecture:** three accounts under AWS Organizations — A ingests via Kinesis with Macie + Lambda tokenization, B hosts Redshift Serverless analytics, C governs via Lake Formation LF-Tags; VPC endpoints keep all traffic off the public internet.
- **Steps:** create the OU structure and attach SCPs; share the Glue catalog cross-account via RAM/Lake Formation; define LF-Tags and tag-based grants; wire Macie findings through EventBridge to the tokenization Lambda; alarm on CloudTrail data events; lock S3/Redshift behind gateway and interface endpoints.
- **Hardest part — the anti-exfiltration SCP:**

```
{
  "Effect": "Deny",
  "Action": ["s3:PutObject", "s3:PutObjectAcl"],
  "Resource": "arn:aws:s3:::/*/*",
  "Condition": {
    "StringNotEquals": {
      "aws:PrincipalAccount": ["111111111111", "222222222222"]
    }
  }
}
```

- **Pitfalls:** a missing cross-account KMS key policy makes shared data unreadable with no obvious error; LF-Tag matching is case-sensitive — a casing mismatch silently denies grants; one interface endpoint per account per AZ adds up fast on the bill.

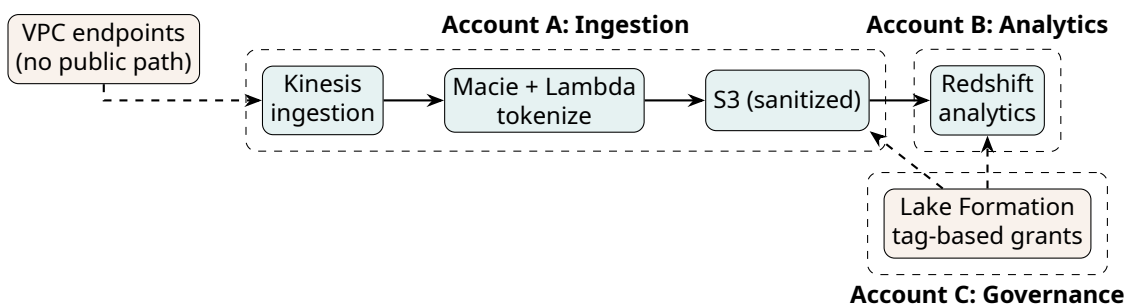


Figure 11: Milestone 6 reference architecture: multi-account governed data mesh

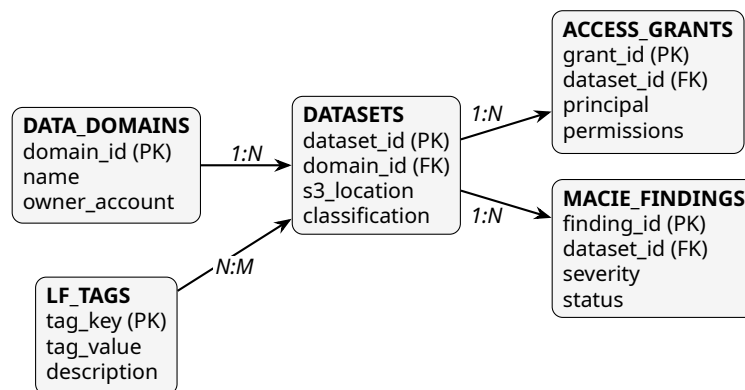


Figure 12: Capstone data model: multi-account governed data mesh

7 Appendix A: Glossary

S3 Storage Class	Durability/cost tier of an S3 object (Standard, IA, Glacier) chosen by access pattern and lifecycle rules.
Object Lock	S3 write-once-read-many protection preventing deletion or overwrite for a retention period.
Aurora Serverless v2	Auto-scaling Aurora configuration that adjusts capacity in ACU increments based on load.
AWS DMS	Managed service for one-time migrations and continuous CDC replication between database engines.
CDC	Change Data Capture — streaming inserts, updates, and deletes from source transaction logs to targets.
RCU/WCU	DynamoDB Read/Write Capacity Units; the billing and throughput model for provisioned tables.
GS	Global Secondary Index — an alternate query key structure on a DynamoDB table with its own throughput.
RA3 node	Redshift node type with managed storage, decoupling compute from data volume.
Distribution Style	Redshift rule (EVEN, KEY, ALL, AUTO) governing how rows are spread across compute slices.
Redshift Spectrum	Redshift feature querying exabytes of S3 data in place via the Glue Data Catalog.
Glue Data Catalog	Central Hive-metastore-compatible metadata store shared by Athena, Redshift, EMR, and Glue.
Lake Formation	Governance layer enforcing database/table/column permissions over S3 data lake objects.
LF-Tag	Key-value attribute in Lake Formation enabling tag-based access control instead of named grants.
DPU	Data Processing Unit — Glue's compute billing unit (4 vCPU, 16 GB) allocated per job.
Job Bookmark	Glue mechanism tracking processed data so re-runs handle only new or changed partitions.
MWAA	Managed Workflows for Apache Airflow — AWS-hosted Airflow with autoscaling workers.
Kinesis Shard	Fixed-throughput unit of a Kinesis stream (1 MB/s in, 2 MB/s out); partition keys map to shards.
Iterator Age	Kinesis consumer lag metric; rising values signal the consumer is falling behind retention.
Watermark	Flink event-time threshold declaring how long late events are accepted into a window.

SPICE	QuickSight's in-memory calculation engine for fast dashboard queries.
Feature Store	SageMaker repository for versioned ML features with online (low-latency) and offline (S3) stores.
Model Registry	SageMaker catalog of versioned, approved model artifacts with deployment lineage.
Model Monitor	SageMaker service detecting data-quality and drift violations on production endpoints.
Macie	ML-driven service discovering and classifying PII and sensitive data in S3.
VPC Endpoint	Private network path to AWS services (Gateway for S3/DynamoDB, Interface via PrivateLink) avoiding the public internet.

8 Appendix B: Assessment & Grading Model

Each monthly capstone is graded on the following weighted rubric:

Category	Weight	What is evaluated
Correctness	30%	Pipeline runs end-to-end and produces the specified outputs from a clean environment.
Reliability & tests	20%	Data-quality tests, failure handling, retries, and a demonstrated failing case.
Security	15%	Least-privilege IAM, encryption, no hard-coded secrets, PII handling.
Cost awareness	15%	Monthly estimate with at least 2 documented optimization decisions.
Documentation	10%	README, architecture diagram, and run-book quality.
Demo	10%	Live or recorded walkthrough of the working system.

The two full-length mock exams scheduled for the weekend after Week 24 feed the final certification readiness score: a consistent score of 80%+ across both mocks indicates readiness to book the AWS Certified Data Engineer – Associate exam; below that, revisit the weak domains noted during Week 24 Friday before booking.

9 Appendix C: Interview Preparation

1. How would you design an S3 lifecycle strategy to minimize cost for a 10PB archive? (Week 1)
2. When would you choose Aurora Serverless v2 over provisioned RDS, and how do you plan failover? (Week 2)
3. Explain single-table design in DynamoDB and when a GSI beats an LSI. (Week 3)
4. A Redshift JOIN is slow — walk through how distribution styles and sort keys would guide your fix. (Week 6)

5. How do Athena, Spectrum, and Redshift differ, and how would you build a governed lake-house combining them? (Weeks 7–8)
6. Glue vs. EMR: how do you choose, and how do job bookmarks guarantee idempotent re-runs? (Weeks 9–10)
7. How do you detect and respond to DMS CDC replication lag before it corrupts downstream reports? (Week 11)
8. Design an Airflow DAG on MWAA with failure alerting; how do CloudWatch alarms and SNS runbooks fit in? (Week 12)
9. Kinesis delivers at-least-once — how do you make consumers idempotent and handle poison records? (Week 13)
10. Describe how you would implement least-privilege, tag-based governance across three AWS accounts with Lake Formation. (Weeks 22–23, Capstone 6)

10 Appendix D: Self-Check Answer Key

Terse answers to the weekly self-check questions.

Week 1:

1. S3 Standard-IA: millisecond retrieval, lower storage price, per-GB retrieval fee.
2. It hides the object; prior versions survive — delete the marker to restore.
3. When bandwidth is insufficient or data is petabyte-scale offline.
4. WORM retention blocks deletion/overwrite, even with stolen credentials.
5. SQS buffers and retries events, decoupling Lambda from burst failures.

Week 2:

1. Multi-AZ: synchronous standby for failover; replicas: asynchronous read scaling.
2. Aurora storage is distributed, six-way replicated across three AZs, self-healing.
3. SCT converts schemas and code; DMS only moves data.
4. Restores always create a new cluster — validates backups without touching production.
5. Replication lag (RPO) and time-to-available (RTO).

Week 3:

1. Skewed keys concentrate throughput on one partition, causing throttling.
2. GSI — addable anytime; LSI only at table creation.
3. It expires permanently; unconsumed stream data is lost.
4. TTL deletes run in the background at no write cost.
5. Single-table fetches related items in one query, avoiding joins.

Week 4:

1. Redis; Memcached is a simple, non-persistent cache.
2. DAX is API-transparent — no client-side cache logic needed.
3. Simultaneous misses hammering the database; jittered TTLs stagger expiry.
4. It caches only what is actually requested, avoiding cold-data writes.
5. Downstream backends from request bursts beyond their capacity.

Week 5:

1. The grain defines what one row means; mixed grains corrupt aggregations.
2. Periodic repeats one row per interval; accumulating updates one row across milestones.
3. When attribute-change history must be preserved for analysis.
4. Auto-Copy: continuous, event-driven ingestion of new S3 files.
5. Spiky workloads with idle periods — no charge while paused.

Week 6:

1. KEY distribution on the join column — collocates matching rows.
2. Interleaved, when queries filter on varying column combinations.
3. Sudden concurrency spikes that would otherwise queue dashboards.

4. When repeated expensive aggregations justify precomputed, refreshable results.
5. Add capacity/concurrency scaling or retune WLM queues.

Week 7:

1. Columnar layout plus compression scans far fewer bytes.
2. Schema and partitions; wrong on inconsistent or deeply nested files.
3. When joining S3 data with Redshift tables in a single query.
4. Athena/Spectrum skip entire S3 prefixes outside the filter.
5. ACID transactions, schema evolution, and time travel on S3.

Week 8:

1. Grants everyone effective super access; revoking enforces Lake Formation control.
2. Tags grant by attribute, scaling without per-table administration.
3. Sensitive PII columns inside otherwise shareable tables.
4. Consumers query the producer's live data read-only, no copies.
5. Queries run without either party seeing the other's raw PII.

Week 9:

1. Task nodes — compute only, no HDFS data to lose.
2. Data persists in S3 independently of cluster lifetime.
3. When standardizing workloads on shared Kubernetes fleets.
4. Installing libraries and configuring nodes before jobs start.
5. You pay per job run instead of for idle cluster hours.

Week 10:

1. Schema handling, `resolveChoice`, and bookmark-aware sources.
2. Re-runs process only new or unprocessed data.
3. Non-urgent jobs on spare capacity at discounted DPU pricing.
4. Serverless and visual — no cluster management for analysts.
5. 4 vCPUs and 16 GB memory, billed per second.

Week 11:

1. `CDCLatencySource` and `CDCLatencyTarget` rising.
2. BACKWARD permits reading newer data (deleting fields); FULL allows both directions.
3. Producer owns the schema; consumers sign off on changes.
4. Bad records are quarantined before they pollute the lake.
5. Breaking: delete a field/change its type; non-breaking: add an optional field.

Week 12:

1. Backfilling hundreds of missed historical runs on deploy.
2. Pure-AWS, event-driven, short-lived workflows without DAG code.
3. Transient failures: throttling, network blips, race conditions.
4. Over-frequent pokes waste API calls and cost; too slow delays the pipeline.

5. Payloads beyond XCom's small size limit — large data goes via S3.

Week 13:

1. 1 MB/s ingress and 2 MB/s egress per shard.
2. Same key routes to the same shard, preserving per-key order.
3. Consumers falling behind before the 24-hour retention expires.
4. Idempotent writes deduplicated on a record ID.
5. Kinesis; MSK is billed per provisioned broker-hour.

Week 14:

1. S3, Redshift, OpenSearch (also Splunk and HTTP endpoints).
2. Enables replay/recovery if transformation or delivery fails.
3. Per-invocation latency plus Lambda GB-second charges.
4. Columnar output slashes downstream Athena/Spectrum scan costs.
5. An S3 error prefix after retries are exhausted.

Week 15:

1. State without TTL grows forever, bloating RocksDB, slowing checkpoints, and causing back-pressure or failure.
2. A savepoint — a manually triggered, versioned snapshot; checkpoints are for automatic failure recovery and may not survive graph changes.
3. The watermark and the join interval: once the watermark passes the interval bound, older state can be dropped.
4. Checkpointing plus a transactional two-phase-commit (or idempotent) sink.
5. When the sink lacks transactions and dedup on a stable record ID is cheaper than exactly-once overhead.

Week 16:

1. Dashboard queries; per-dataset in-memory capacity, requiring refresh.
2. Dataset rules filter rows per user or group at query time.
3. Syncing curated warehouse data back into CRM, marketing, and support tools.
4. The same person across systems must resolve to one identity first.
5. Freshness needs vs. API rate limits, cost, and warehouse load.

Week 17:

1. Training/AutoML to SageMaker Autopilot; inference stays in SQL.
2. Zero data movement and fast SQL-only iteration.
3. The inference function name and signature used in queries.
4. F1 balances precision and recall on the minority churn class.
5. Custom algorithms, GPUs, or dedicated real-time endpoints needed.

Week 18:

1. Serverless endpoints — they scale to zero between requests.

2. The transformation recipe as code or pipeline steps.
3. Custom frameworks, dependencies, or non-built-in algorithms.
4. Trains and tunes many candidates, ranking by validation metric.
5. Enforces governed, auditable access instead of uncontrolled copies.

Week 19:

1. Offline store (S3) for training; online store for low-latency inference.
2. Only approved model versions become deployable.
3. Data drift: input distribution shifts; concept drift: the target relationship changes.
4. Drift may mean broken ingestion — retraining on corrupt data is worse.
5. Baseline statistics and constraints computed from training data.

Week 20:

1. Textract; Comprehend analyzes text semantics, not layout.
2. Serverless, event-driven processing with no servers to manage.
3. Large documents exceed synchronous payload/timeout limits.
4. Index the extracted JSON into OpenSearch.
5. OCR text may contain PII — encrypt, classify, and restrict access.

Week 21:

1. The Gateway endpoint; interface endpoints bill per AZ-hour plus GB.
2. The maximum permissions the role can ever obtain.
3. Rotation shrinks the exposure window of any leaked credential.
4. SCPs bound entire accounts/OUs; IAM policies grant within them.
5. The public internet — traffic stays on the AWS backbone.

Week 22:

1. The job can halt the load and raise an SNS alert.
2. Grants follow tags across accounts without per-table administration.
3. To trace the failure to the upstream table that introduced it.
4. `ColumnExists` catches drift; `Completeness` catches nulls.
5. The domain team that publishes the data product.

Week 23:

1. ML-based classification of sensitive data types and context.
2. Object Lock preserves evidence immutably; deletion is irreversible.
3. Tokenization (via the token vault); hashing is one-way.
4. Object-level API calls such as `GetObject` and `PutObject`.
5. Over-broad regexes flood findings and train teams to ignore alerts.

Week 24:

1. Prevents concurrent state corruption and enables team/CI collaboration.
2. Business rules: uniqueness, accepted values, referential relationships.
3. Isolates experiments; production receives only tested code.
4. Short-lived tokens — no long-lived keys to leak from secrets.
5. dbt tests pass and the Terraform plan is clean.

11 Appendix E: Datasets & Project Data

Prefer AWS-hosted open data from the Registry of Open Data on AWS (<https://registry.opendata.aws>) — they sit in public S3 buckets next to your compute, so no egress cost or download step.

- **NYC Taxi & Limousine Trip Records** (<https://registry.opendata.aws/nyc-tlc-trip-records-pds/>, s3://nyc-tlc/): billions of trip rows in CSV/Parquet. Used in Week 5 COPY benchmarking, the Week 6 distribution-style benchmark, and the Milestone 2 lakehouse capstone.
- **NOAA Global Historical Climatology Network (GHCN)** (<https://registry.opendata.aws/ghcn/>, s3://noaa-ghcn-pds/): daily weather observations from 100k+ stations. Feeds the Milestone 3 batch ELT capstone and Week 15 late-arriving-data exercises.
- **Olist Brazilian E-Commerce dataset** (<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>): 100k orders across customers, sellers, products, payments, and reviews — a natural star schema. Used in the Milestone 1 e-commerce capstone and the Week 5 dimensional-modeling exercises; mirror the CSVs into your own S3 bucket.

Synthetic data: generate customers, orders, and clickstream with Python Faker (<https://github.com/joke2k/faker>); the Week 13 producer script shows the pattern for streaming events. Expected volumes per milestone: Milestone 1 ~1M rows in Aurora/DynamoDB; Milestone 2 ~10M rows as Parquet in S3; Milestone 3 ~50M raw events per batch; Milestone 4 ~5k events/minute sustained; Milestone 5 ~100k feature rows; Milestone 6 small governed samples (~100k rows) with injected PII.

12 Appendix F: Portfolio & Presentation Guide

Each capstone should ship as a public GitHub repository you can walk an interviewer through. **README structure (in this order):**

1. **Problem:** one paragraph — who hurts, at what scale, and what “done” means.
2. **Architecture diagram first:** reviewers decide in 10 seconds; lead with the picture.
3. **Setup:** exact commands to reproduce from a clean environment (prereqs, IaC, seed data).
4. **Screenshots:** working dashboard, pipeline run, and one alarm firing.
5. **Cost estimate:** monthly table with the two optimization decisions you made.
6. **Lessons learned:** one failure you hit and how you fixed it.

Resume bullet templates (two per milestone — fill in the brackets):

- **M1:** Built a multi-tier e-commerce data backend on Aurora Serverless, DynamoDB, and S3 with DMS CDC replication, reconciling [N] rows/day with automated data-quality tests.
- **M1:** Designed multi-region S3 lifecycle and DR strategy, cutting projected storage spend [X]% via Glacier transitions and validated restores.
- **M2:** Built a governed lakehouse (S3/Parquet, Glue Catalog, Lake Formation column-level security) serving Redshift Spectrum dashboards to [N] analysts.
- **M2:** Benchmarked Athena vs. Spectrum vs. Redshift Serverless, documenting a cost decision saving ~\$[X]/month at [N] TB scanned.
- **M3:** Automated a batch ELT pipeline with EventBridge and Step Functions orchestrating Glue data-quality gates and EMR Serverless Spark into Redshift.

- **M3:** Authored MWAA (Airflow) DAGs with idempotent Glue bookmarks and SNS runbook alerting, eliminating duplicate loads on re-runs.
- **M4:** Built a real-time fraud-detection stream (Kinesis → Flink sliding windows → Redshift → QuickSight) flagging cross-city card use within 10 minutes.
- **M4:** Implemented idempotent Kinesis consumers with DLQ routing, sustaining [N] events/minute with zero duplicate alerts.
- **M5:** Productionized an MLOps pipeline: Athena feature extraction → SageMaker Feature Store → Pipelines → Model Registry with Model Monitor drift alarms.
- **M5:** Gated auto-retraining behind metric thresholds and human approval, catching [N] injected drift violations before deployment.
- **M6:** Deployed a three-account governed data mesh with Lake Formation LF-Tags, Macie tokenization quarantine, and VPC endpoint lockdown via Terraform.
- **M6:** Built GitHub Actions CI/CD with OIDC federation and dbt tests gating Dev→Prod promotion for [N] data models.

Talking about your project in interviews: open with the problem and its scale (“10M rows nightly, 500 analysts”), then show the architecture diagram and narrate two design decisions with their trade-offs — interviewers score reasoning, not service names. Have one failure story ready (what broke, how you detected it, what you changed). Close with numbers: cost per month, latency, row counts. If asked “what would you do differently,” you pass only if you have a real answer — write it in the README’s lessons-learned section while the project is fresh.

13 6-Month Progress Tracking Timetable

Week	Primary Focus	Key Project / Capstone
1	Amazon S3 & Migration	S3 Lifecycle & DataSync
2	Amazon RDS & Aurora	Aurora Serverless Failover Simulation
3	Amazon DynamoDB	DynamoDB Single-Table Design & GS
4	Hybrid Ingestion & Caches	Cap 1: E-Commerce Multi-Tier Data
5	Redshift Architecture & Dimensional Modeling	COPY from S3 to Redshift Serverless star-schema load
6	Redshift Optimization	Dist/Sort Keys Benchmark
7	Athena & Lakehouse	Glue Crawler & Athena vs Spectrum
8	AWS Lake Formation	Cap 2: Enterprise Data Lakehouse
9	Amazon EMR	Spark Job on EMR Cluster
10	AWS Glue ETL	Glue Studio Visual Pipeline
11	AWS DMS, EventBridge & Data Contracts	RDS to S3 CDC via DMS
12	Amazon MWAA (Airflow)	Cap 3: Automated Batch ELT
13	Amazon Kinesis Data Streams	IoT Producer/Consumer Scripting
14	Kinesis Data Firehose	Firehose inline Lambda format conversion
15	Managed Apache Flink	Flink Tumbling Windows on Streams
16	Amazon QuickSight & Reverse ETL	Cap 4: Real-Time Fraud Telemetry
17	Redshift ML	Churn Prediction SQL Models
18	SageMaker Foundations	Autopilot Deployment & Endpoint Te
19	SageMaker Pipelines	SageMaker Pipeline execution
20	AI APIs (Textract/Rekognition)	Cap 5: End-to-End MLOps Pipeline
21	IAM & VPC Endpoints	S3 Gateway Endpoint implementation
22	Glue Data Quality	Automated DQ Rulesets & Alerts
23	Amazon Macie	PII Discovery & Lambda Quarantine
24	Production DataOps (IaC, dbt & CI/CD) & Certification Prep	Cap 6: Secure Data Mesh Design Terraform/dbt repo with CI/CD

"Consistency is the architecture of mastery."

Best of luck on your 6-month journey to becoming a Master AWS Data Engineer!